
LIMTIC

Production-Ready Backend API Blueprint

Research Laboratory Management Platform

Complete database models, REST endpoints, authentication scheme, and CRUD operations derived from the Figma-generated frontend.

Stack: Node.js / Express or NestJS · PostgreSQL · Prisma/TypeORM · JWT · S3-compatible storage

Equally compatible with: Django REST Framework, Spring Boot, FastAPI, Laravel

1. Executive Overview

LIMTIC is a research laboratory management platform with a public-facing informational site and a role-based back office. The Figma frontend exposes **10 public pages** and **5 authenticated dashboards** (one per role). All data currently lives as static mocks inside React components; this blueprint translates every static section into a concrete database model, REST endpoint, and authorization rule a backend team can implement directly.

Roles & Permissions Matrix

Role	Code	Primary Capabilities
Super Admin	SUPER_ADMIN	Full system access: user CRUD, all content CRUD, audit log, system settings, statistics
Admin	ADMIN	Same as Super Admin minus: user role changes, audit log, system settings
Chercheur (Researcher)	CHERCHEUR	Own profile, own publications CRUD, supervise students, view team/events
Doctorant (PhD student)	DOCTORANT	Own profile, own publications (submit for validation), view team/events/axes
Mastérien (Master student)	MASTERIEN	Own profile, own publications (submit for validation), view team/events/axes

Public vs Authenticated Surface

Surface	Routes	Auth
Public site	/, /equipe, /chercheurs/:id, /publications, /publications/:id, /axes-recherche, /axes-recherche/:id, /evenements, /evenements/:id, /contact	None (read-only)
Auth flow	/login, /forgot-password	None
Dashboard	/dashboard/{role}/*	JWT (role-gated)

Cross-Cutting Backend Concerns

- **Auth:** JWT access token (15 min) + refresh token (7 days, http-only cookie or rotating refresh). Role embedded in token claims.
- **Password policy:** ≥ 8 chars, bcrypt cost 12, password-reset tokens single-use 30 min.
- **Pagination:** cursor or page+limit on every list endpoint (default *page=1*, *limit=20*, max limit=100).
- **Search:** case-insensitive substring on *title*, *authors*, *name*, *email* via Postgres trigram (pg_trgm) or ILIKE.
- **Filtering:** any list endpoint accepts query params matching the frontend SearchFilter component groups.
- **File uploads:** multipart/form-data $\rightarrow$ S3-compatible bucket. Limits: avatar 5 MB (jpg/png/webp), PDF 20 MB, event photos 10 MB each.
- **i18n:** all user-facing enum labels are translated client-side; the API returns stable codes (e.g. *ARTICLE_JOURNAL*, *A_VENIR*).
- **Soft delete:** users, publications, events carry a *deletedAt* timestamp; hard-delete is Super Admin only.
- **Audit log:** every CREATE/UPDATE/DELETE on Users, Publications, Events, Axes, Settings writes a row to AuditLog.

- **Rate limiting:** 100 req/min per IP for public endpoints; 300 req/min per user for authenticated; 5 req/min for /auth/login.
- **CORS:** allowlist the front-end origin only.
- **Email delivery:** SMTP credentials live in SystemSettings (configurable by Super Admin) — used for password reset, contact form, notification digests.

Convention

Every protected endpoint requires header **Authorization: Bearer <jwt>**. Allowed roles are listed under each endpoint as *Auth*. **403 Forbidden** is returned when the token is valid but the role is insufficient; **401 Unauthorized** when the token is missing/expired.

2. Database Models & Entities

Models are presented in a database-agnostic form. All tables include `id` (UUID, PK), `createdAt`, `updatedAt`; soft-deletable tables add `deletedAt` (nullable). Foreign keys are marked **FK**. Required fields are **bold**.

2.1 User

Stores all platform accounts regardless of role. Profile-specific fields (thesis title, master speciality, encadrant link, etc.) are kept on this table since the frontend's ProfileForm renders a single union form whose visible fields depend on the role.

Field	Type	Notes
email	string, unique	login identifier
passwordHash	string	bcrypt
firstName	string	
lastName	string	
role	enum	SUPER_ADMIN ADMIN CHERCHEUR DOCTORANT MASTERIEN
active	bool	soft-toggle (login blocked when false)
avatarUrl	string	S3 URL — uploaded via /me/avatar
phone	string	
bio	text	shown on public researcher page
grade	enum / string	Chercheur only: Professeur Maître de Conférences Assistant
speciality	string	Chercheur main domain
office	string	office number / building
orcid	string	ORCID identifier
googleScholar	url	
researchGate	url	
linkedin	url	
website	url	
thesisTitle	string	Doctorant only
encadrantId	uuid FK → User	Doctorant/Mastérien thesis director
enrollmentYear	int	Doctorant: year of registration
masterSpeciality	string	Mastérien only
projectTitle	string	Mastérien: PFE / project title
lastLoginAt	datetime	

Field	Type	Notes
deletedAt	datetime	soft delete

2.2 Axe (Research Axis)

Field	Type	Notes
title	string	e.g. 'Intelligence Artificielle et Apprentissage Automatique'
description	text	
color	string	tailwind color key for card stripe
responsibleId	uuid FK → User	must be CHERCHEUR
themes	string[]	array of theme tags (Machine Learning, NLP...)
order	int	display order on AxesPage

Junction: AxeMember — (axeId FK, userId FK, role enum: RESPONSABLE | CHERCHEUR | DOCTORANT | MASTERIEN). Allows a member to belong to multiple axes.

2.3 Publication

Field	Type	Notes
type	enum	ARTICLE_JOURNAL CONFERENCE_INT CONFERENCE_NAT CHAPITRE_OUVRAGE RAPPORT_TECHNIQUE
title	string	
year	int	
abstract	text	
keywords	string[]	free-form tags
visibility	enum	PUBLIQUE PRIVEE
status	enum	BROUILLON SOUMIS PUBLIE REJETE — workflow state
rejectionReason	text	filled when status=REJETE
doi	string	indexed unique
pdfUrl	string	S3 URL
submittedById	uuid FK → User	the author who created the entry
validatedById	uuid FK → User	Admin who approved
—— Journal fields ——		when type=ARTICLE_JOURNAL
journalName	string	
volume	string	
issue	string	
pages	string	e.g. '123-145'
impactFactor	decimal	
quartile	enum	Q1 Q2 Q3 Q4
snip	decimal	
issn	string	
—— Conference fields ——		when type=CONFERENCE_*
conferenceName	string	
acronym	string	e.g. 'CCS 2026'
location	string	
country	string	
coreRanking	enum	A* A B C
indexation	string[]	['Scopus', 'WoS']
—— Book chapter ——		when type=CHAPITRE_OUVRAGE

Field	Type	Notes
bookTitle	string	
editor	string	
isbn	string	
—— Tech report ——		when type=RAPPORT_TECHNIQUE
reportNumber	string	
institution	string	

Junctions:

- **PublicationAuthor** — (publicationId FK, userId FK nullable, externalName string nullable, externalAffiliation string nullable, externalEmail string nullable, order int). Supports both internal (linked user) and external co-authors.
- **PublicationAxe** — (publicationId FK, axelId FK). A publication can belong to multiple axes.

2.4 Event

Field	Type	Notes
type	enum	SEMINAIRE ATELIER CONFERENCE JOURNEE_ETUDE
title	string	
startDate	datetime	
endDate	datetime	nullable for single-day events
location	string	
description	text	
coverImageUrl	string	hero banner
status	enum (computed)	A_VENIR EN_COURS PASSE — derived from dates
agenda	json	ordered array: [{time, title, description, type}]
createdById	uuid FK→User	

Children:

- **EventSpeaker** — (eventId FK, name, affiliation, role enum: CONFERENCIER | MODERATEUR | INTERVENANT, talkTitle, bio, photoUrl, order int).
- **EventPhoto** — (eventId FK, url, caption, uploadedById FK→User, order int). Only relevant when event is PASSE.
- **EventRegistration** — optional (eventId FK, name, email, affiliation, registeredAt). Used if the public site adds an inscription form to upcoming events.

2.5 Encadrement (Supervision)

Represents the supervisor → student relationship displayed on the Chercheur's *Encadrements* dashboard and on the public researcher profile *Encadrements* tab.

Field	Type	Notes
supervisorId	uuid FK→User	must be CHERCHEUR
studentId	uuid FK→User	DOCTORANT or MASTERIEN
type	enum	DOCTORAT MASTER (mirrors student role)
subject	text	thesis / project title
abstract	text	
startYear	string	e.g. '2024' or '2025-2026'
status	enum	EN_COURS SOUTENU DIPLOME
defenseDate	date	nullable, set when status changes

2.6 Notification

Field	Type	Notes
recipientId	uuid FK→User	who sees it
type	enum	PUBLICATION_VALIDATED PUBLICATION_REJECTED PUBLICATION_SUBMITTED ENCADREMENT_ADDED EVENT_CREATED USER_CREATED SYSTEM
title	string	short heading
message	text	body shown in the card
link	string	deep-link inside the app (e.g. /publications/123)
readAt	datetime	null when unread
relatedEntityType	string	e.g. 'Publication'
relatedEntityId	uuid	

2.7 ContactMessage

Stores submissions from the public Contact page form. Admins/SuperAdmins can list, mark read, and reply.

Field	Type	Notes
name	string	
email	string	
subject	string	
message	text	
status	enum	NEW READ REPLIED SPAM

Field	Type	Notes
handledById	uuid FK → User	admin who processed it
ipAddress	string	for spam mitigation

2.8 AuditLog

Field	Type	Notes
actorId	uuid FK → User	who performed the action
action	enum	CREATE UPDATE DELETE LOGIN LOGOUT VALIDATE REJECT ROLE_CHANGE
resourceType	string	User Publication Event Axe Setting Encadrement
resourceId	uuid	
changes	json	{ before: {...}, after: {...} } diff
ipAddress	string	
userAgent	string	

AuditLog rows are immutable — no UPDATE, no DELETE endpoints.

2.9 SystemSetting

Singleton row (or key-value table) for global configuration edited on the Super Admin Settings page.

Field	Type	Notes
labName	string	default 'LIMTIC'
slogan	string	
contactEmail	string	
contactPhone	string	
address	text	
socialLinks	json	{ linkedin, twitter, ... }
smtpHost	string	
smtpPort	int	
smtpUser	string	
smtpPasswordEnc	string	encrypted at rest
smtpFromEmail	string	
logoUrl	string	S3 URL
faviconUrl	string	

2.10 PasswordResetToken

Field	Type	Notes
userId	uuid FK → User	
tokenHash	string	sha256 of the token sent by email
expiresAt	datetime	30 minutes from issue
usedAt	datetime	single-use marker

3. Authentication & Account Endpoints

Endpoints backing `LoginPage.tsx`, `ForgotPasswordPage.tsx`, the `AuthContext` mock *login/logout*, and the profile dropdown in `DashboardLayout.tsx`.

POST	<code>/api/v1/auth/login</code>
Used by	LoginPage form submit (line 16 of <code>LoginPage.tsx</code>). Replaces the mock <code>setUser()</code> in <code>AuthContext</code> .
Auth	Public · rate-limit 5/min/IP

Request body

```
{
  "email": "ahmed.bensalem@limtic.tn",
  "password": ".....",
  "rememberMe": true
}
```

Response (200/201)

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "user": {
    "id": "uuid",
    "email": "ahmed.bensalem@limtic.tn",
    "firstName": "Ahmed",
    "lastName": "Ben Salem",
    "role": "CHERCHEUR",
    "avatarUrl": "https://..."
  }
}
```

Notes: Wrong credentials → 401 (frontend `toast.error`). Inactive account → 403. The frontend uses `user.role` to choose redirect target — that logic moves server-side via the `role` claim.

POST	<code>/api/v1/auth/refresh</code>
Used by	Silent token refresh from the SPA's interceptor.
Auth	Public · refresh-token cookie or body

Request body

```
{ "refreshToken": "eyJhbGciOi..." }
```

Response (200/201)

```
{ "accessToken": "eyJhbGciOi...", "refreshToken": "eyJhbGciOi..." }
```

Notes: Implement rotating refresh tokens — invalidate the prior token on use.

POST	/api/v1/auth/logout
Used by	DashboardLayout 'Se déconnecter' button.
Auth	Authenticated

Response (200/201)

```
{ "ok": true }
```

Notes: Invalidate the refresh token on the server side. Frontend clears its AuthContext.

POST	/api/v1/auth/forgot-password
Used by	ForgotPasswordPage submit.
Auth	Public · rate-limit 3/min/IP

Request body

```
{ "email": "ahmed.bensalem@limtic.tn" }
```

Response (200/201)

```
{ "ok": true }
```

Notes: Always returns 200 to avoid email enumeration. If the address exists, send a reset link with a 30-min single-use token (PasswordResetToken table).

POST	/api/v1/auth/reset-password
Used by	Reset-password landing page (linked from the email).
Auth	Public (token-gated)

Request body

```
{  
  "token": "from email URL",  
  "newPassword": "....."  
}
```

Response (200/201)

```
{ "ok": true }
```

Notes: Marks PasswordResetToken.usedAt and updates User.passwordHash.

GET	/api/v1/me
Used by	App boot — hydrates AuthContext from a persisted access token.
Auth	Authenticated

Response (200/201)

```
{ "id": "...", "email": "...", "firstName": "...", "lastName": "...", "role": "CHERCHEUR",  
  "avatarUrl": "...", ... }
```

Notes: Returns the full User row minus sensitive fields (passwordHash never serialized).

PUT	/api/v1/me
Used by	ProfileForm.handleSaveProfile() — every role's /profile page.
Auth	Authenticated (any role, own profile only)

Request body

```
{
  "firstName": "Ahmed", "lastName": "Ben Salem",
  "phone": "+216...", "bio": "...",
  "grade": "Professeur", "speciality": "IA",
  "office": "B12",
  "orcid": "0000-0002-...",
  "googleScholar": "https://...",
  "researchGate": "https://...",
  "linkedin": "https://...",
  "website": "https://...",
  "thesisTitle": "...", // doctorant
  "encadrantId": "uuid", // doctorant/masterien
  "enrollmentYear": 2024,
  "masterSpeciality": "...", "projectTitle": "..."
}
```

Response (200/201)

```
{ updated user object }
```

Notes: The server ignores any field the role is not entitled to fill (e.g. a CHERCHEUR cannot set *thesisTitle*). Email and role changes are **not** permitted here — separate admin endpoint.

POST	/api/v1/me/avatar
Used by	ProfileForm photo picker (input type=file, <5 MB).
Auth	Authenticated · multipart/form-data

Request body

```
Content-Type: multipart/form-data
file: <image binary> // jpg | png | webp · ≤ 5 MB
```

Response (200/201)

```
{ "avatarUrl": "https://cdn..../avatars/uuid.webp" }
```

Notes: Server resizes to 512×512, strips EXIF, stores in S3, updates User.avatarUrl.

PUT	/api/v1/me/password
Used by	ProfileForm.handleChangePassword().
Auth	Authenticated

Request body

```
{ "currentPassword": "...", "newPassword": "≥ 8 chars" }
```

Response (200/201)

```
{ "ok": true }
```

Notes: Verifies currentPassword first. Front already validates length and confirmation match (lines 67-79 of ProfileForm).

4. Public Site Endpoints (no authentication)

Read-only endpoints powering the marketing/informational pages. All responses must be cache-friendly (set *Cache-Control: public, max-age=60, stale-while-revalidate=300*) and never include unpublished/private content.

4.1 HomePage · /

Sections requiring data: *Statistics bar* (4 counters), *Axes de Recherche* (top 3 cards), *Publications Récentes* (3 newest), *Agenda* (3 upcoming events).

GET	/api/v1/public/stats
Used by	HomePage StatCounter components (line 90-93).
Auth	Public

Response (200/201)

```
{
  "researchers": 24,
  "publications": 156,
  "doctorants": 18,
  "events": 32
}
```

Notes: Aggregate counts: researchers = User.count(role=CHERCHEUR, active=true); publications = Publication.count(status=PUBLIE); doctorants = User.count(role=DOCTORANT, active=true); events = Event.count().

GET	/api/v1/public/axes
Used by	HomePage Axes section + full AxesPage list.
Auth	Public
Params	?limit=3 (HomePage uses 3; AxesPage omits)

Response (200/201)

```
[
  {
    "id": "uuid",
    "title": "Intelligence Artificielle",
    "description": "...",
    "color": "teal",
    "responsible": {
      "id": "uuid", "firstName": "Ahmed", "lastName": "Ben Salem",
      "avatarUrl": "..."
    },
    "themes": ["Machine Learning", "NLP", "Computer Vision"],
    "memberCount": 4,
    "publicationCount": 45
  }
]
```

GET	/api/v1/public/publications/recent
Used by	HomePage Publications Récentes (3 cards).
Auth	Public
Params	?limit=3

Response (200/201)

```
[ { full publication object – see 4.3 } ]
```

Notes: Returns the most recent PUBLIE + PUBLIQUE publications ordered by year DESC, createdAt DESC.

GET	/api/v1/public/events/upcoming
Used by	HomePage Agenda section (line 182-198).
Auth	Public
Params	?limit=3

Response (200/201)

```
[ { "id": "...", "type": "SEMINAIRE", "title": "...",
  "startDate": "2026-06-15T09:00:00Z",
  "location": "Amphithéâtre A, ISI", "status": "A_VENIR" } ]
```

4.2 TeamPage · /equipe

Three tabs: Chercheurs, Doctorants, Mastériens. The active tab can be reflected as a query param so the page is shareable.

GET	/api/v1/public/members
Used by	TeamPage ChercheursList / DoctorantsList / MasteriensList components.
Auth	Public
Params	role=CHERCHEUR DOCTORANT MASTERIEN (required) · page, limit

Response (200/201)

```
{
  "items": [
    // For CHERCHEUR:
    { "id": "1", "firstName": "Ahmed", "lastName": "Ben Salem",
      "grade": "Professeur", "speciality": "Intelligence Artificielle",
      "email": "ahmed.bensalem@isi.utm.tn", "avatarUrl": "..." },
    // For DOCTORANT:
    { "id": "...", "firstName": "Sarah", "lastName": "Trabelsi",
      "thesisTitle": "Apprentissage profond...",
      "encadrant": { "id": "...", "firstName": "Ahmed", "lastName": "Ben Salem" },
      "enrollmentYear": 2024, "status": "EN_COURS", "avatarUrl": "..." },
    // For MASTERIEN:
    { "id": "...", "firstName": "Ines", "lastName": "Hamdi",
      "projectTitle": "Système de recommandation...",
      "encadrant": { ... }, "enrollmentYear": "2025-2026",
      "status": "EN_COURS", "avatarUrl": "..." }
  ],
  "total": 24, "page": 1, "limit": 20
}
```

Notes: status for students comes from the latest Encadrement record where studentId matches.

4.3 ResearcherPage • /chercheurs/:id

GET	/api/v1/public/researchers/:id
Used by	ResearcherPage header (lines 17-39) + Bio tab (lines 59-66).
Auth	Public

Response (200/201)

```
{
  "id": "uuid",
  "firstName": "Ahmed", "lastName": "Ben Salem",
  "grade": "Professeur", "speciality": "Intelligence Artificielle",
  "email": "ahmed.bensalem@isi.utm.tn",
  "phone": "+216 71 123 456",
  "bio": "Dr. Ahmed Ben Salem est professeur...",
  "avatarUrl": "...",
  "orcid": "0000-...", "googleScholar": "https://...",
  "researchGate": "...", "linkedin": "...", "website": "...",
  "axes": [ { "id": "...", "title": "..." } ]
}
```

Notes: 404 when the user does not exist or is not role CHERCHEUR or not active.

GET	/api/v1/public/researchers/:id/publications
Used by	ResearcherPage Publications tab (line 69-83).
Auth	Public
Params	page, limit, year, type, ranking

Response (200/201)

Same shape as 4.4 list (paginated).

Notes: Filters by Publication where this user is in PublicationAuthor and status=PUBLIE and visibility=PUBLIQUE.

GET	/api/v1/public/researchers/:id/encadrements
Used by	ResearcherPage Encadrements tab (line 85-98).
Auth	Public

Response (200/201)

```
[
  { "id": "...", "type": "DOCTORAT",
    "student": { "id": "...", "firstName": "Sarah", "lastName": "Trabelsi", "avatarUrl": "...",
    "subject": "Apprentissage profond...",
    "status": "EN_COURS", "startYear": "2024" }
  }
]
```

4.4 PublicationsPage • /publications

Heavy filter+search page. Maps to the *SearchFilter* component's filter groups (type, ranking, year, axe) seen at line 72-110 of PublicationsPage.tsx.

GET	/api/v1/public/publications
Used by	PublicationsPage list + counters in the header (line 154-158).
Auth	Public
Params	q search title / authors / venue type ARTICLE_JOURNAL CONFERENCE_INT ... (repeatable) ranking q1 q2 q3 q4 core-a-star ... (repeatable) year int (repeatable) axeId uuid (repeatable) sort year_desc (default) year_asc createdAt_desc page, limit

Response (200/201)

```
{
  "items": [
    { "id": "1", "type": "ARTICLE_JOURNAL",
      "title": "Deep Learning for Medical Imaging...",
      "year": 2026, "ranking": "q1",
      "authors": ["A. Ben Salem", "F. Gharbi", "S. Trabelsi"],
      "venue": "IEEE Trans. Medical Imaging",
      "doi": "10.1109/TMI.2026.123456",
      "axes": [{ "id": "...", "title": "Intelligence Artificielle..." } ] }
  ],
  "total": 156,
  "stats": { "total": 156, "journals": 89, "conferences": 67 },
  "page": 1, "limit": 20
}
```

Notes: *stats* ignores pagination and is computed across the full filtered result. *ranking* in the response is a derived field combining quartile (for journals) and coreRanking (for conferences).

GET	/api/v1/public/publications/:id
Used by	PublicationDetailPage (entire body of the file).
Auth	Public

Response (200/201)

```
{
  "id": "1", "type": "ARTICLE_JOURNAL", "title": "...",
  "year": 2026, "abstract": "...", "keywords": ["Deep Learning", ...],
  "authors": [
    { "userId": "uuid", "name": "Ahmed Ben Salem", "external": false },
    { "userId": null, "name": "Marie Dubois", "affiliation": "Hôpital...", "external": true }
  ],
  "axes": [ { "id": "...", "title": "..." } ],
  "doi": "10.1109/TMI.2026.123456",
  "pdfUrl": "https://cdn.../pubs/uuid.pdf",
  // type-specific block (journal here):
  "journalName": "IEEE Trans...", "volume": "45", "issue": "3",
  "pages": "123-145", "impactFactor": 10.245,
  "quartile": "Q1", "issn": "0278-0062"
}
```

4.5 AxesPage · /axes-recherche/[:id]

GET /api/v1/public/axes (see 4.1) backs the list. For the detail page below, an additional endpoint is required.

GET	/api/v1/public/axes/:id
Used by	AxisDetailPage — replaces the static <i>axesData</i> object.
Auth	Public

Response (200/201)

```
{
  "id": "1",
  "title": "Intelligence Artificielle et Apprentissage Automatique",
  "description": "...",
  "color": "accent-blue",
  "responsible": { "id": "...", "firstName": "Ahmed", "lastName": "Ben Salem", "email": "..."},
  "themes": ["Machine Learning", "Deep Learning", "NLP", ...],
  "members": [
    { "id": "...", "firstName": "...", "lastName": "...", "role": "RESPONSABLE" },
    { "id": "...", "firstName": "...", "lastName": "...", "role": "DOCTORANT" }
  ],
  "publicationCount": 45
}
```

4.6 EventsPage • /evenements/:id

GET	/api/v1/public/events
Used by	EventsPage tabs A_VENIR / EN_COURS / PASSE (line 27).
Auth	Public
Params	status=A_VENIR EN_COURS PASSE, type, page, limit, q

Response (200/201)

```
{
  "items": [
    { "id": "1", "type": "SEMINAIRE", "title": "IA et Santé",
      "startDate": "2026-06-15T09:00:00Z",
      "endDate": "2026-06-15T17:00:00Z",
      "location": "Amphithéâtre A, ISI",
      "status": "A_VENIR",
      "description": "...",
      "speakerCount": 5, "photoCount": 0, "hasPhotos": false,
      "coverImageUrl": "..." }
  ],
  "total": 24, "page": 1, "limit": 20
}
```

Notes: *status* is computed server-side from *startDate*/*endDate* vs. *now()*.

GET	/api/v1/public/events/:id
Used by	EventDetailPage description, programme, speakers list and photo gallery.
Auth	Public

Response (200/201)

```
{
  "id": "1", "type": "SEMINAIRE", "title": "...", "description": "...",
  "startDate": "...", "endDate": "...", "location": "...",
  "status": "PASSE", "coverImageUrl": "...",
  "agenda": [
    { "time": "09h00 - 10h00", "title": "Accueil et inscription", "description": "Café..." },
    { "time": "10h00 - 11h30", "title": "Conférence inaugurale", "description": "..." }
  ],
  "speakers": [
    { "name": "Dr. Ahmed Ben Salem", "affiliation": "LIMTIC, ISI",
      "role": "CONFERENCIER", "talkTitle": "Deep Learning...",
      "photoUrl": "..." }
  ],
  "photos": [
    { "id": "...", "url": "https://...", "caption": "..." }
  ]
}
```

Notes: *photos* array is empty unless status=PASSE and there are uploaded photos. Drives the PhotoGallery component.

POST	/api/v1/public/events/:id/register
Used by	Optional public registration form on upcoming events (visible in EventDetailPage sidebar).
Auth	Public · captcha + rate-limit 3/hour/IP

Request body

```
{ "name": "...", "email": "...", "affiliation": "..." }
```

Response (200/201)

```
{ "ok": true, "registrationId": "uuid" }
```

Notes: Sends a confirmation email. Creates EventRegistration row. 422 if event is not A_VENIR.

4.7 ContactPage · /contact

POST	/api/v1/public/contact
Used by	ContactPage form submit (handleSubmit, line 12).
Auth	Public · captcha + rate-limit 5/hour/IP

Request body

```
{
  "name": "...", "email": "...",
  "subject": "...", "message": "..."
}
```

Response (200/201)

```
{ "ok": true }
```

Notes: Stores ContactMessage row, sends notification email to contactEmail in SystemSettings, returns success → frontend shows toast.success('Message envoyé...').

GET	/api/v1/public/lab-info
Used by	ContactPage right column + PublicFooter (address, phone, social).
Auth	Public

Response (200/201)

```
{ "labName": "LIMTIC", "contactEmail": "...", "contactPhone": "...", "address": "...",
  "socialLinks": { "linkedin": "...", "twitter": "..." } }
```

5. Dashboard — Publications

Backs the publication management pages for every role: *SuperAdminPublications.tsx*, *ChercheurPublications.tsx*, *DoctorantPublications.tsx*, *MasterienPublications.tsx*, plus the *DoctorantAllPublications* / *MasterienAllPublications* read-only views, and the *PublicationForm* wizard.

Validation Workflow

Doctorants and Mastériens cannot publish directly. They create publications in **BROUILLON** status; on submit, the status becomes **SOUMIS** and the publication appears in the Admin/SuperAdmin queue. Admins/SuperAdmins move it to **PUBLIE** (visible publicly) or **REJETE** (with a reason that triggers a notification back to the author). Chercheurs may transition straight to PUBLIE (*canPublishDirectly=true* in *PublicationForm.tsx*).

GET	/api/v1/publications
Used by	All dashboard publication lists. Admins see the full table; researchers see filtered-by-author by default.
Auth	Authenticated (any role)
Params	scope mine (default for researchers) all (admin only) status BROUILLON SOUMIS PUBLIE REJETE type, ranking, year, axeId, q page, limit, sort

Response (200/201)

Same shape as 4.4 plus `<i>status, visibility, submittedBy, validatedBy, rejectionReason</i>`.

Notes: Non-admins receive 403 if they request *scope=all*. Researchers' default scope filters to publications where they are listed in *PublicationAuthor*.

POST	/api/v1/publications
Used by	PublicationForm Submit button (wizard final step).
Auth	CHERCHEUR · DOCTORANT · MASTERIEN · ADMIN · SUPER_ADMIN

Request body

```
{
  "type": "ARTICLE_JOURNAL",
  "title": "Deep Learning...", "year": 2026,
  "abstract": "...", "keywords": ["Deep Learning", "Medical"],
  "visibility": "PUBLIQUE",
  "axeIds": ["uuid", "uuid"],
  "authors": [
    { "userId": "uuid" }, // internal
    { "name": "Marie Dubois", "affiliation": "Hôpital",
      "email": "marie@..." } // external
  ],
  // type-specific block (journal):
  "journalName": "IEEE Trans...", "volume": "45",
  "issue": "3", "pages": "123-145", "doi": "10.1109/...",
  "impactFactor": 10.245, "quartile": "Q1", "snip": 2.1,
  "issn": "0278-0062",
  "status": "SOUMIS" // researchers can also pass "BROUILLON"; admins can pass "PUBLIE"
}
```

Response (200/201)

```
{ created publication object with computed id }
```

Notes: Server enforces the type-specific required fields (e.g. journalName for ARTICLE_JOURNAL, conferenceName + coreRanking for CONFERENCE_INT). DOI uniqueness is checked across all publications.

POST	/api/v1/publications/:id/pdf
Used by	PublicationForm PDF upload field (formData.pdfFile, line 50).
Auth	Author or Admin · multipart

Request body

```
Content-Type: multipart/form-data
file: <pdf binary> · ≤ 20 MB
```

Response (200/201)

```
{ "pdfUrl": "https://cdn.../pubs/uuid.pdf" }
```

Notes: PDF is virus-scanned. Stored in S3 with a content-addressable path. The Download PDF button on PublicationDetailPage links to this URL.

GET	/api/v1/publications/:id
Used by	PublicationForm 'edit' mode pre-fill; admin detail modal.
Auth	Authenticated — author OR admin

Response (200/201)

```
Same as 4.4 detail + internal fields (status, submittedBy, validatedBy, rejectionReason).
```

PUT /api/v1/publications/:id**Used by** PublicationForm 'edit' mode submit.**Auth** Author (only while status=BROUILLON/REJETE) · Admin · SuperAdmin**Request body**

Same shape as POST.

Response (200/201)

{ updated publication }

Notes: A researcher cannot edit a PUBLIE publication — only an Admin can.**DELETE** /api/v1/publications/:id**Used by** SuperAdminPublications row action (Trash2 icon) and the role-specific Publications tables.**Auth** Author (only while status=BROUILLON) · Admin · SuperAdmin**Response (200/201)**

{ "ok": true }

Notes: Soft delete by default. Triggers AuditLog DELETE. ConfirmDialog on the frontend already gates the action client-side.**POST** /api/v1/publications/:id/submit**Used by** Researcher 'Soumettre pour validation' button (DOCTORANT / MASTERIEN dashboards).**Auth** DOCTORANT · MASTERIEN (author only)**Response (200/201)**

{ "id": "...", "status": "SOUMIS" }

Notes: Transitions BROUILLON → SOUMIS. Creates a PUBLICATION_SUBMITTED notification for every active ADMIN and SUPER_ADMIN.**POST** /api/v1/publications/:id/validate**Used by** SuperAdmin/Admin Publications page approval action.**Auth** ADMIN · SUPER_ADMIN**Request body**

{ "visibility": "PUBLIQUE" } // optional override

Response (200/201)

{ "id": "...", "status": "PUBLIE", "validatedBy": "..." }

Notes: Triggers PUBLICATION_VALIDATED notification for every author.

POST	/api/v1/publications/:id/reject
Used by	SuperAdmin/Admin Publications page rejection action.
Auth	ADMIN · SUPER_ADMIN

Request body

```
{ "reason": "Résumé insuffisant. Veuillez détailler la méthodologie." }
```

Response (200/201)

```
{ "id": "...", "status": "REJETE", "rejectionReason": "..." }
```

Notes: Triggers PUBLICATION_REJECTED notification carrying the reason — surfaced by the NotificationCard on the author's dashboard.

6. Dashboard — Events

Powers *SuperAdminEvents.tsx* (full CRUD) and the read-only Events pages for every other role.

GET	/api/v1/events
Used by	All dashboard Events pages.
Auth	Authenticated
Params	status, type, q, page, limit

Response (200/201)

Same shape as 4.6 list.

POST	/api/v1/events
Used by	SuperAdminEvents 'Nouvel événement' modal.
Auth	ADMIN · SUPER_ADMIN

Request body

```
{
  "type": "SEMINAIRE",
  "title": "IA et Santé",
  "startDate": "2026-06-15T09:00:00Z",
  "endDate": "2026-06-15T17:00:00Z",
  "location": "Amphithéâtre A, ISI",
  "description": "...",
  "agenda": [
    { "time": "09h00 - 10h00", "title": "Accueil...", "description": "..." }
  ],
  "speakers": [
    { "name": "...", "affiliation": "...", "role": "CONFERENCIER",
      "talkTitle": "...", "bio": "..." }
  ]
}
```

Response (200/201)

```
{ created event }
```

Notes: *status* is computed, never sent by the client.

PUT	/api/v1/events/:id
Used by	SuperAdminEvents 'edit' modal.
Auth	ADMIN · SUPER_ADMIN

Request body

Same as POST.

Response (200/201)

```
{ updated event }
```

DELETE /api/v1/events/:id

Used by	SuperAdminEvents row trash action (with ConfirmDialog).
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

```
{ "ok": true }
```

Notes: Soft delete; cascades to EventSpeakers, EventPhotos, EventRegistrations on hard-delete only.

POST /api/v1/events/:id/cover

Used by	Event form 'Image de couverture' upload (Upload icon visible in SuperAdminEvents.tsx).
Auth	ADMIN · SUPER_ADMIN · multipart

Request body

```
file: <image> · ≤ 5 MB
```

Response (200/201)

```
{ "coverImageUrl": "https://..." }
```

Speakers sub-resource

POST /api/v1/events/:id/speakers

Used by	Add-speaker UI inside the event modal.
Auth	ADMIN · SUPER_ADMIN

Request body

```
{ "name": "...", "affiliation": "...", "role": "CONFERENCIER", "talkTitle": "...", "bio": "..." }
```

Response (200/201)

```
{ created speaker }
```

PUT /api/v1/events/:id/speakers/:speakerId

Used by	Edit-speaker action.
Auth	ADMIN · SUPER_ADMIN

Request body

```
Same fields.
```

Response (200/201)

```
{ updated speaker }
```

DELETE	/api/v1/events/:id/speakers/:speakerId
---------------	--

Used by	Remove-speaker (X) icon.
----------------	--------------------------

Auth	ADMIN · SUPER_ADMIN
-------------	---------------------

Response (200/201)

```
{ "ok": true }
```

Photos sub-resource

Photo gallery only appears for PASSE events on the public EventDetailPage (PhotoGallery component). Admins upload after the event.

POST	/api/v1/events/:id/photos
-------------	---------------------------

Used by	Event admin gallery uploader (Image / Upload icons in SuperAdminEvents).
----------------	--

Auth	ADMIN · SUPER_ADMIN · multipart
-------------	---------------------------------

Request body

```
Content-Type: multipart/form-data
files[]: <image> (multiple, each ≤ 10 MB)
captions[]: "string" (optional, parallel array)
```

Response (200/201)

```
[ { "id": "...", "url": "...", "caption": "..." } ]
```

Notes: Server creates thumbnails (300, 800, 1600 px). The PhotoGallery uses the 800px version; the lightbox uses 1600px.

DELETE	/api/v1/events/:id/photos/:photoId
---------------	------------------------------------

Used by	Photo card delete (X) button.
----------------	-------------------------------

Auth	ADMIN · SUPER_ADMIN
-------------	---------------------

Response (200/201)

```
{ "ok": true }
```

Registrations

GET	/api/v1/events/:id/registrations
------------	----------------------------------

Used by	Admin event detail panel (optional 'inscrits' tab).
----------------	---

Auth	ADMIN · SUPER_ADMIN
-------------	---------------------

Params	page, limit, q
---------------	----------------

Response (200/201)

```
{ "items": [ { name, email, affiliation, registeredAt } ], "total": 42 }
```

GET	/api/v1/events/:id/registrations/export
Used by	'Exporter CSV' button.
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

CSV file (Content-Type: text/csv)

7. Dashboard — Axes de Recherche

Backs SuperAdminAxes.tsx (CRUD), and the read-only Axes views for Chercheur / Doctorant / Mastérien dashboards.

GET	/api/v1/axes
Used by	Every Axes dashboard page.
Auth	Authenticated

Response (200/201)

Same as 4.1, plus the count of pending publications waiting on this axe.

POST	/api/v1/axes
Used by	SuperAdminAxes 'Nouvel axe' modal.
Auth	ADMIN · SUPER_ADMIN

Request body

```
{
  "title": "Big Data et Analyse de Données",
  "description": "...",
  "color": "success",
  "responsibleId": "uuid",           // must be CHERCHEUR
  "themes": ["Data Mining", "Visualization", "ETL"],
  "memberIds": ["uuid", "uuid"]    // optional initial members
}
```

Response (200/201)

```
{ created axe }
```

PUT	/api/v1/axes/:id
Used by	SuperAdminAxes edit action.
Auth	ADMIN · SUPER_ADMIN

Request body

Same shape as POST.

Response (200/201)

```
{ updated axe }
```

DELETE	/api/v1/axes/:id
Used by	SuperAdminAxes trash action.
Auth	SUPER_ADMIN

Response (200/201)

```
{ "ok": true }
```

Notes: 422 if any publication still references this axe.

POST	/api/v1/axes/:id/members
Used by	'Ajouter un membre' button on axe detail.
Auth	ADMIN · SUPER_ADMIN

Request body

```
{ "userId": "uuid", "role": "CHERCHEUR" }
```

Response (200/201)

```
{ "ok": true }
```

DELETE	/api/v1/axes/:id/members/:userId
Used by	Remove-member action.
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

```
{ "ok": true }
```

8. Dashboard — User Management

Backs *SuperAdminUsers.tsx* — Super Admin and Admin can list, create, edit, activate/deactivate, and delete users. Only Super Admin can change a user's role.

GET	/api/v1/users
Used by	SuperAdminUsers table + counters.
Auth	ADMIN · SUPER_ADMIN
Params	role SUPER_ADMIN ADMIN CHERCHEUR DOCTORANT MASTERIEN status active inactive q search firstName/lastName/email page, limit, sort

Response (200/201)

```
{
  "items": [
    { "id": "1", "firstName": "Super", "lastName": "Admin",
      "email": "superadmin@limtic.tn", "role": "SUPER_ADMIN",
      "active": true, "createdAt": "2024-01-10",
      "lastLoginAt": "2026-05-23",
      "specialization": "Intelligence Artificielle" }
  ],
  "total": 9,
  "counts": { "SUPER_ADMIN": 1, "ADMIN": 1, "CHERCHEUR": 3, "DOCTORANT": 2, "MASTERIEN": 2 },
  "page": 1, "limit": 20
}
```

POST	/api/v1/users
Used by	SuperAdminUsers 'Nouvel utilisateur' modal.
Auth	ADMIN · SUPER_ADMIN (Admin cannot create SUPER_ADMIN)

Request body

```
{
  "firstName": "Karim", "lastName": "Mansouri",
  "email": "karim@limtic.tn",
  "password": "≥ 8 chars",           // optional – server can also generate + email
  "role": "ADMIN",
  "specialization": "..."          // maps to speciality (CHERCHEUR) or masterSpeciality, etc.
}
```

Response (200/201)

```
{ created user (no passwordHash) }
```

Notes: If *password* is omitted, the server generates one and emails it together with a one-time password-reset link.

PUT /api/v1/users/:id

Used by	SuperAdminUsers edit modal.
---------	-----------------------------

Auth	ADMIN · SUPER_ADMIN
------	---------------------

Request body

Same fields as POST, password optional.

Response (200/201)

```
{ updated user }
```

Notes: Role change: only SUPER_ADMIN may submit a *role* field — Admins changing role get 403.

PATCH /api/v1/users/:id/status

Used by	SuperAdminUsers ToggleLeft/ToggleRight icon.
---------	--

Auth	ADMIN · SUPER_ADMIN
------	---------------------

Request body

```
{ "active": false }
```

Response (200/201)

```
{ "id": "...", "active": false }
```

Notes: Inactive users cannot log in (login returns 403).

DELETE /api/v1/users/:id

Used by	SuperAdminUsers trash action (with ConfirmDialog).
---------	--

Auth	SUPER_ADMIN
------	-------------

Response (200/201)

```
{ "ok": true }
```

Notes: Soft delete. The user's authored publications stay; the PublicationAuthor row keeps a snapshot of the display name.

POST /api/v1/users/:id/reset-password

Used by	Optional 'Réinitialiser mot de passe' admin action.
---------	---

Auth	ADMIN · SUPER_ADMIN
------	---------------------

Response (200/201)

```
{ "ok": true }
```

Notes: Sends a password-reset email to the user as if they had clicked 'forgot password'.

9. Dashboard — Encadrements (Supervision)

Backs *ChercheurEncadrements.tsx* — the supervisor's view of their PhD students and Master students.

GET	/api/v1/encadrements
Used by	ChercheurEncadrements tabs (doctorants/masteriens).
Auth	CHERCHEUR (own) · ADMIN · SUPER_ADMIN
Params	supervisorId (default=me), type=DOCTORAT MASTER, status

Response (200/201)

```
[
  {
    "id": "1",
    "type": "DOCTORAT",
    "student": {
      "id": "...",
      "firstName": "Sarah",
      "lastName": "Trabelsi",
      "email": "...",
      "avatarUrl": "..."
    },
    "subject": "Apprentissage profond...",
    "abstract": "...",
    "status": "EN_COURS",
    "startYear": "2024",
    "publicationCount": 3
  }
]
```

Notes: *publicationCount* is computed via *PublicationAuthor* where *userId* = *student.id* and *status* = *PUBLIE*.

POST	/api/v1/encadrements
Used by	'Ajouter un encadrement' modal (admin / chercheur).
Auth	CHERCHEUR (as supervisor) · ADMIN · SUPER_ADMIN

Request body

```
{
  "studentId": "uuid",
  "type": "DOCTORAT",
  "subject": "...",
  "abstract": "...",
  "startYear": "2024"
}
```

Response (200/201)

```
{ created encadrement }
```

Notes: Triggers *ENCADREMENT_ADDED* notification to the student.

PUT	/api/v1/encadrements/:id
Used by	Edit modal.
Auth	Supervisor · Admin · SuperAdmin

Request body

```
{ "subject": "...", "abstract": "...", "status": "SOUTENU", "defenseDate": "2026-09-15" }
```

Response (200/201)

```
{ updated encadrement }
```

DELETE	/api/v1/encadrements/:id
Used by	Remove action.
Auth	Supervisor · Admin · SuperAdmin

Response (200/201)

```
{ "ok": true }
```

10. Dashboard — Notifications

Backs every role's *Notifications* page (admin/chercheur/doctorant/masterien/superadmin) and the bell icon in *DashboardLayout.tsx*.

GET	/api/v1/notifications
Used by	Notifications pages list + bell-dropdown preview.
Auth	Authenticated
Params	unreadOnly=true false, type, page, limit

Response (200/201)

```
{
  "items": [
    {
      "id": "...", "type": "PUBLICATION_VALIDATED",
      "title": "Publication validée",
      "message": "Votre publication 'Deep Learning...' a été validée.",
      "link": "/dashboard/chercheur/publications",
      "readAt": null,
      "createdAt": "2026-05-23T12:00:00Z",
      "relatedEntityType": "Publication", "relatedEntityId": "uuid"
    }
  ],
  "unreadCount": 5,
  "total": 23, "page": 1, "limit": 20
}
```

Notes: *unreadCount* drives the red badge on the bell icon.

GET	/api/v1/notifications/unread-count
Used by	Bell-icon badge polling / SSE fallback.
Auth	Authenticated

Response (200/201)

```
{ "unreadCount": 5 }
```

POST	/api/v1/notifications/:id/read
Used by	Clicking a notification card.
Auth	Authenticated (recipient only)

Response (200/201)

```
{ "id": "...", "readAt": "2026-05-23T12:34:56Z" }
```

POST	/api/v1/notifications/read-all
Used by	'Tout marquer comme lu' button (line 10 of ChercheurNotifications).
Auth	Authenticated

Response (200/201)

```
{ "marked": 5 }
```

DELETE /api/v1/notifications/:id

Used by	Optional dismiss action.
Auth	Authenticated (recipient only)

Response (200/201)

```
{ "ok": true }
```

Real-time delivery

- Recommended: **WebSocket** at /ws/notifications (JWT-authenticated, room = userId). Push the full notification object on creation.
- Fallback: **Server-Sent Events** at GET /api/v1/notifications/stream.
- Worst case: client polls *unread-count* every 30 s.

Notification trigger map

Event	Notification type	Recipients
Doctorant/Mastérien submits publication	PUBLICATION_SUBMITTED	All ADMIN + SUPER_ADMIN
Admin validates publication	PUBLICATION_VALIDATED	All authors of that publication
Admin rejects publication	PUBLICATION_REJECTED	All authors (carries rejectionReason)
Supervisor adds an encadrement	ENCADREMENT_ADDED	The student
Admin creates a new event	EVENT_CREATED	Optional: all active users
Admin creates a user account	USER_CREATED	The new user (welcome notif + email)
SuperAdmin posts a system-wide announcement	SYSTEM	All users

11. Statistics, Audit Log & System Settings

11.1 Statistics

Backs *SuperAdminStatistics.tsx* — bar chart (publications/year) and pie chart (publications by type), powered by Recharts.

GET	/api/v1/statistics/overview
Used by	Statistics dashboard top KPIs.
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

```
{
  "users": { "total": 35, "byRole": { "CHERCHEUR": 12, "DOCTORANT": 14, "MASTERIEN": 7,
  "ADMIN": 1, "SUPER_ADMIN": 1 } },
  "publications": { "total": 156, "byStatus": { "PUBLIE": 132, "SOUMIS": 8, "BROUILLON": 16 }
},
  "events": { "total": 32, "upcoming": 5, "past": 27 },
  "axes": { "total": 4 }
}
```

GET	/api/v1/statistics/publications-by-year
Used by	BarChart (line 4-10 of Statistics.tsx).
Auth	ADMIN · SUPER_ADMIN
Params	from=2022&to=2026

Response (200/201)

```
[ { "year": 2022, "count": 24 }, { "year": 2023, "count": 32 }, ... ]
```

GET	/api/v1/statistics/publications-by-type
Used by	PieChart (line 12-16 of Statistics.tsx).
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

```
[ { "type": "ARTICLE_JOURNAL", "label": "Journaux", "count": 89 }, { "type":
"CONFERENCE_INT", "label": "Conf. Int.", "count": 45 }, ... ]
```

GET	/api/v1/statistics/publications-by-axe
Used by	Future drill-down chart (also useful on Axe detail).
Auth	ADMIN · SUPER_ADMIN

Response (200/201)

```
[ { "axeId": "...", "title": "Intelligence Artificielle", "count": 45 }, ... ]
```

11.2 Audit Log

Backs *SuperAdminAudit.tsx*. SUPER_ADMIN exclusive. AuditLog rows are append-only.

GET	/api/v1/audit-logs
Used by	Audit table on the SuperAdmin dashboard.
Auth	SUPER_ADMIN
Params	action CREATE UPDATE DELETE LOGIN VALIDATE REJECT ROLE_CHANGE resourceType User Publication Event Axe Setting Encadrement actorId uuid from / to ISO timestamps q free text on the changes JSON page, limit

Response (200/201)

```
{
  "items": [
    {
      "id": "...",
      "timestamp": "2026-05-22T14:30:00Z",
      "actor": { "id": "...", "email": "admin@limtic.tn" },
      "action": "CREATE",
      "resourceType": "Publication", "resourceId": "123",
      "changes": { "after": { "title": "..." } },
      "ipAddress": "41.x.x.x" }
  ],
  "total": 1234, "page": 1, "limit": 20
}
```

GET	/api/v1/audit-logs/export
Used by	'Exporter' button (recommended addition).
Auth	SUPER_ADMIN

Response (200/201)

```
CSV stream
```

11.3 System Settings

Backs *SuperAdminSettings.tsx* — lab identity, SMTP configuration, branding.

GET	/api/v1/settings
Used by	Page load.
Auth	SUPER_ADMIN

Response (200/201)

```
{ labName, slogan, contactEmail, contactPhone, address, socialLinks, smtpHost, smtpPort,
  smtpUser, smtpFromEmail, logoUrl, faviconUrl }
```

Notes: `smtpPassword` is never returned — only an indicator `smtpPasswordConfigured: true`.

PUT /api/v1/settings**Used by** 'Sauvegarder les modifications' button.**Auth** SUPER_ADMIN**Request body**Partial of the GET response, plus optional `<i>smtpPassword</i>` (write-only).**Response (200/201)**

{ updated settings }

Notes: On password change, encrypt at rest with the app's master key.**POST** /api/v1/settings/test-smtp**Used by** 'Tester la configuration' button (line 44 of Settings.tsx).**Auth** SUPER_ADMIN**Request body**

{ "to": "verify@example.com" } // optional, defaults to current user email

Response (200/201)

{ "ok": true, "messageId": "..." }

Notes: Sends a single test email synchronously; returns the SMTP error message on failure.**POST** /api/v1/settings/logo**Used by** Logo upload (recommended addition to the Settings page).**Auth** SUPER_ADMIN · multipart**Request body**

file: <png|svg>

Response (200/201)

{ "logoUrl": "..." }

12. File Upload Surface

Consolidated table of every upload endpoint. All uploads use *multipart/form-data*, are virus-scanned, and store objects in an S3-compatible bucket behind a signed-URL CDN. EXIF and other identifying metadata are stripped from images.

Endpoint	Frontend trigger	Accepted	Max size	Bucket path
POST /me/avatar	ProfileForm photo	jpg, png, webp	5 MB	avatars/{userId}.webp
POST /publications/:id/pdf	PublicationForm wizard	pdf	20 MB	publications/{pubId}.pdf
POST /events/:id/cover	SuperAdminEvents form	jpg, png, webp	5 MB	events/cover/{eventId}.webp
POST /events/:id/photos	Event gallery uploader	jpg, png, webp (multi)	10 MB each	events/{eventId}/{photoId}.webp
POST /settings/logo	System settings (extension)	png, svg	2 MB	settings/logo.png

Storage tips

Use a presigned-URL flow for large files: client requests *POST /uploads/sign*, uploads directly to S3, then calls *POST /publications/:id/pdf/confirm* with the object key. This keeps the API server out of the upload path entirely.

13. Error Response Contract

All errors follow a single shape so the frontend's `toast.error()` calls and form validators can rely on a consistent envelope.

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Le titre est obligatoire",
    "fields": {
      "title": "Le titre est obligatoire",
      "year": "L'année doit être supérieure à 1900"
    }
  }
}
```

HTTP	code	When
400	VALIDATION_ERROR	Body fails JSON schema / business rules
401	UNAUTHENTICATED	Missing / invalid / expired access token
403	FORBIDDEN	Token valid, role insufficient
404	NOT_FOUND	Resource does not exist (or soft-deleted)
409	CONFLICT	Duplicate email, duplicate DOI, etc.
422	UNPROCESSABLE	Business rule violation (e.g. delete axe with publications)
429	RATE_LIMITED	Endpoint-specific limit exceeded
500	INTERNAL_ERROR	Bug — log with correlation id

Standard headers

- **X-Request-Id** — UUID, echoed back; useful in support tickets.
- **X-RateLimit-Remaining, X-RateLimit-Reset** — populated on rate-limited endpoints.
- **Cache-Control** — public endpoints get *public, max-age=60*; authenticated get *private, no-store*.

14. Frontend ↔ Endpoint Coverage Matrix

Every Figma-generated page mapped to the endpoints it consumes. Use this as the implementation checklist when wiring real data into the React app.

14.1 Public site

Page / Component	Endpoints
HomePage.tsx	GET /public/stats · GET /public/axes?limit=3 · GET /public/publications/recent?limit=3 · GET /public/events/upcoming?limit=3
TeamPage.tsx (3 tabs)	GET /public/members?role=CHERCHEUR · ?role=DOCTORANT · ?role=MASTERIEN
ResearcherPage.tsx (3 tabs)	GET /public/researchers/:id · /publications · /encadrements
PublicationsPage.tsx	GET /public/publications (with SearchFilter params)
PublicationDetailPage.tsx	GET /public/publications/:id
AxesPage.tsx	GET /public/axes
AxisDetailPage.tsx	GET /public/axes/:id
EventsPage.tsx (3 tabs)	GET /public/events?status=A_VENIR · =EN_COURS · =PASSE
EventDetailPage.tsx	GET /public/events/:id · POST /public/events/:id/register (optional)
ContactPage.tsx	POST /public/contact · GET /public/lab-info
PublicFooter.tsx	GET /public/lab-info (cached)
LoginPage.tsx	POST /auth/login
ForgotPasswordPage.tsx	POST /auth/forgot-password

14.2 Dashboard chrome

Component	Endpoints
DashboardLayout — boot	GET /me
DashboardLayout — bell icon	GET /notifications/unread-count + WS /ws/notifications
DashboardLayout — bell dropdown	GET /notifications?unreadOnly=true&limit=5
DashboardLayout — profile menu logout	POST /auth/logout
DashboardLayout — theme toggle	Frontend-only (ThemeContext); persist to /me PATCH if cross-device sync needed
DashboardLayout — language toggle	Frontend-only (LanguageContext); persist to /me PATCH if needed

14.3 Super Admin

Page	Endpoints
Overview	GET /statistics/overview
Users	GET / POST / PUT / PATCH /:id/status / DELETE /users · POST /:id/reset-password
Publications	GET /publications?scope=all · POST /:id/validate · /reject · DELETE
Events	Full CRUD + speakers + photos + registrations export
Axes	Full CRUD + members
Statistics	GET /statistics/overview · /publications-by-year · /publications-by-type · /publications-by-axe
Audit	GET /audit-logs · /export
Settings	GET / PUT /settings · POST /settings/test-smtp · /settings/logo
Notifications	GET /notifications · POST /:id/read · /read-all
Profile	GET / PUT /me · POST /me/avatar · PUT /me/password

14.4 Admin

Same endpoints as Super Admin minus: audit log, system settings, user role changes, axe deletion.

14.5 Chercheur

Page	Endpoints
Overview	GET /me/dashboard (aggregator endpoint — see note below)
Profile	GET / PUT /me · POST /me/avatar · PUT /me/password
Publications	GET /publications?scope=mine · POST · PUT · DELETE · /pdf upload
Encadrements	GET /encadrements · POST · PUT · DELETE
Axes (read)	GET /axes
Notifications	Standard notification endpoints
Events (read)	GET /events
Team (read)	GET /public/members + supervisor's team filter

Aggregator endpoint suggestion: GET /me/dashboard returns a single object with counts (my publications by status, my encadrements by status, unread notifications, upcoming events I'm registered to) — avoids 5+ round-trips at every dashboard load.

14.6 Doctorant

Page	Endpoints
Overview	GET /me/dashboard
Profile	GET / PUT /me · POST /me/avatar · PUT /me/password
Publications (mine)	GET /publications?scope=mine · POST · PUT (BROUILLON only) · /submit · DELETE
AllPublications (read)	GET /public/publications
Axes (read)	GET /axes
Team (read)	GET /public/members
Events / Notifications	Standard endpoints

14.7 Mastérien

Identical surface to Doctorant — only difference is the role on the user record and the labels in the navigation sidebar (see DashboardLayout.tsx case 'MASTERIEN').

15. Security & Implementation Checklist

Pre-launch checklist for the backend implementation team.

Authentication & sessions

- Bcrypt cost ≥ 12 for password hashing.
- Access tokens 15 min, refresh tokens 7 days with rotation; revoke on logout and password change.
- Account-lockout after 10 failed logins within 15 minutes.
- Email-based password reset tokens single-use, hashed at rest, 30-min TTL.
- Optional: 2FA TOTP for ADMIN and SUPER_ADMIN roles.

Authorization

- Centralize role checks in a middleware / guard; never duplicate the rule per endpoint.
- Field-level authorization: a researcher updating `/me` cannot set `role`, `email`, or another user's `encadrantId`.
- Resource-level authorization: every `:id` route must check ownership *before* loading the resource (avoid IDOR).
- Soft-deleted rows are invisible to non-admin queries — filter at the ORM layer.

Input validation

- Validate every request body with a schema (Joi, Zod, class-validator, pydantic, etc.).
- Enforce enum closure server-side — never trust client-sent enums.
- DOI/ISSN/ORCID format validation with regex; reject otherwise with 400.
- Strip HTML from all free-text fields before storage; render with an XSS-safe component on the frontend.
- Sanitize filenames; never use the user-provided name on disk.

Data protection

- Encrypt SMTP password and any other secret at rest using a KMS-backed key.
- TLS 1.2+ everywhere; HSTS preload.
- Personal data export (GDPR) — endpoint `GET /me/export` returning JSON of every row referencing the user.
- Personal data deletion — SUPER_ADMIN exclusive, hard-delete with PublicationAuthor snapshot.

Observability

- Structured logs with X-Request-Id and userId on every line.
- Metrics: request rate, error rate, p50/p95/p99 latency per route.
- Audit log shipped to long-term cold storage monthly.
- Health endpoint: `GET /healthz` (no auth, returns DB + S3 + SMTP probe results).

Performance

- Indexes on: User.email (unique), Publication.doi (unique), Publication.status, Publication.year, Event.startDate, Notification.recipientId+readAt, AuditLog.actorId+timestamp.
- Postgres trigram index on Publication.title, User.lastName for fast ILIKE search.
- Caching layer (Redis) for `/public/*` endpoints — invalidate on relevant mutations.
- CDN in front of `/public/*` and all S3-backed asset URLs.
- Pagination with cursor (id-based) on AuditLog and Notification — these tables grow fast.

CI/CD & testing

- Contract tests using OpenAPI + Schemathesis (or equivalent).
- E2E tests for the four workflows: login flow, publication validation flow, event creation with photo upload, role-change by SUPER_ADMIN.
- Database migrations versioned and reversible.
- Seeds for the 5 test accounts shown on LoginPage.tsx (superadmin@, admin@, chercheur@, doctorant@, masterien@limtic.tn) — for staging only.

Appendix A — OpenAPI 3 Skeleton

Starter *openapi.yaml* stub the team can expand. Listed paths align exactly with the endpoint sections of this document.

```
openapi: 3.0.3
info:
  title: LIMTIC API
  version: 1.0.0
servers:
  - url: https://api.limtic.tn/api/v1

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

security:
  - bearerAuth: []

paths:
  /auth/login: { post: {...} }
  /auth/refresh: { post: {...} }
  /auth/logout: { post: {...} }
  /auth/forgot-password: { post: {...} }
  /auth/reset-password: { post: {...} }

  /me: { get: {...}, put: {...} }
  /me/avatar: { post: {...} }
  /me/password: { put: {...} }
  /me/dashboard: { get: {...} }

  /public/stats: { get: { security: [] } }
  /public/axes: { get: { security: [] } }
  /public/axes/{id}: { get: { security: [] } }
  /public/members: { get: { security: [] } }
  /public/researchers/{id}: { get: { security: [] } }
  /public/researchers/{id}/publications: { get: { security: [] } }
  /public/researchers/{id}/encadrements: { get: { security: [] } }
  /public/publications: { get: { security: [] } }
  /public/publications/recent: { get: { security: [] } }
  /public/publications/{id}: { get: { security: [] } }
  /public/events: { get: { security: [] } }
  /public/events/upcoming: { get: { security: [] } }
  /public/events/{id}: { get: { security: [] } }
  /public/events/{id}/register: { post: { security: [] } }
  /public/contact: { post: { security: [] } }
  /public/lab-info: { get: { security: [] } }

  /users: { get: {...}, post: {...} }
  /users/{id}: { get: {...}, put: {...}, delete: {...} }
  /users/{id}/status: { patch: {...} }
  /users/{id}/reset-password: { post: {...} }

  /publications: { get: {...}, post: {...} }
  /publications/{id}: { get: {...}, put: {...}, delete: {...} }
  /publications/{id}/pdf: { post: {...} }
  /publications/{id}/submit: { post: {...} }
  /publications/{id}/validate: { post: {...} }
  /publications/{id}/reject: { post: {...} }
```

```

/events: { get: {...}, post: {...} }
/events/{id}: { get: {...}, put: {...}, delete: {...} }
/events/{id}/cover: { post: {...} }
/events/{id}/speakers: { post: {...} }
/events/{id}/speakers/{spId}: { put: {...}, delete: {...} }
/events/{id}/photos: { post: {...} }
/events/{id}/photos/{phId}: { delete: {...} }
/events/{id}/registrations: { get: {...} }
/events/{id}/registrations/export: { get: {...} }

/axes: { get: {...}, post: {...} }
/axes/{id}: { put: {...}, delete: {...} }
/axes/{id}/members: { post: {...} }
/axes/{id}/members/{userId}: { delete: {...} }

/encadrements: { get: {...}, post: {...} }
/encadrements/{id}: { put: {...}, delete: {...} }

/notifications: { get: {...} }
/notifications/unread-count: { get: {...} }
/notifications/{id}/read: { post: {...} }
/notifications/read-all: { post: {...} }
/notifications/{id}: { delete: {...} }

/statistics/overview: { get: {...} }
/statistics/publications-by-year: { get: {...} }
/statistics/publications-by-type: { get: {...} }
/statistics/publications-by-axe: { get: {...} }

/audit-logs: { get: {...} }
/audit-logs/export: { get: {...} }

/settings: { get: {...}, put: {...} }
/settings/test-smtp: { post: {...} }
/settings/logo: { post: {...} }

/healthz: { get: { security: [] } }

```

Endpoint count

~ **65 distinct endpoints** implementing every dynamic feature in the Figma frontend. The team can use the OpenAPI skeleton above as the single source of truth: generate server stubs (Express, NestJS, FastAPI, Spring) and TypeScript client types from the same file, then plug each route into the React component listed in Section 14.

End of blueprint